

Day 54 – JUnit (Day 01)

Why Reflection API is Useful Before Learning JUnit

Class Note: These notes are for revision only. Follow them along with the classroom explanation and code demonstration.

What We Covered Today

- Problem with a normal Java testing approach.
- Why this approach becomes difficult for student evaluation.
- How the Reflection API solves those problems.
- How this concept connects to JUnit Testing.

Version 1 – Normal Testing Approach

In class, we first created a simple test program that directly calls the student's method.

Student Solution Code

This is the same code used during the classroom demonstration.

```
package com.nit.project;

/*
 * solution code through which student can take the help
 * or verify their ans after submitting
 */
public class Calculation {

    public int sum(int a,int b) {
        return a+b;
    }

}

// ----- submit
*/

//Prefilled code which student will write there ans
public class Calculation {

    public int sum(int a,int b) {
```

```
        int result=a+b;
        return result;
    }
}
```

Version 1 Test Code

```
package com.nit.project;

/*
 * Test case code which check the student code which
 * they will write in prefilled code
 */
public class CalculationTest {

    public static void main(String[] args) {

        Calculation calculation=new Calculation();

        if((calculation.sum(10, 20)==30 &&
            calculation.sum(-10,-20)==-30) &&
            calculation.sum(0, 0)==0) {

            System.out.println("All Test case passed");
        }
        else {

            System.out.println("Test case fail");
            System.out.println("class not found");
            System.out.println("Parameter not matched");
        }
    }
}
```

Why This Approach Creates Problems

During the live demonstration, we intentionally changed different parts of the student's code and observed what happened.

Change Made	What Happened
Class name changed	Test could not work as expected.
Method name changed	Method could not be found.
Parameters changed	Signature mismatch occurred.
Only output checked	Program structure was never verified.

Key Observation

The test mainly checks the final output, but it does not properly verify whether the student followed the required structure.

Version 2 – Reflection API Based Testing

Instead of directly depending on the class, we moved to a runtime-based approach using Reflection.

What Reflection Checks

- Class Name
- Method Name
- Parameter Types
- Constructor
- Return Type
- Output Values

Notice how each validation happens separately. This is the important difference discussed in class.

Reflection API Test Code

```
package com.nit.project;

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

/*
 * What are challenges you
 *
 * Unit Testing
 * Junit Testing
 */

public class CalculationTestVersion2 {

    public static void main(String[] args)
```

```

        throws IllegalAccessException, IllegalArgumentException,
        InvocationTargetException {

    /*
     * 1st Load the class at runtime
     */

    Class<?> forName = null;

    try {
        forName = Class.forName("com.nit.project.Calculation");
    } catch (ClassNotFoundException e) {
        System.err.println("Class not found or defined");
    }

    /*
     * 2nd match the name of the class
     */

    if (!forName.getSimpleName().equals("Calculation")) {
        System.out.println("Class name not matched!");
    }

    /*
     * Let check the method name and parameter
     */

    Method declaredMethod = null;

    try {
        declaredMethod = forName.getDeclaredMethod("sum", int.class,
int.class);
    } catch (NoSuchMethodException e) {
        System.err.println("Method name not found ! could be name, parameter
missmtached!");
    } catch (SecurityException e) {
        e.printStackTrace();
    }

    /*
     * create the object
     *
     */

    Object newInstance = null;

    try {

```

```

        newInstance = forName.getDeclaredConstructor().newInstance();
    } catch (InstantiationException | IllegalAccessException |
IllegalArgumentException
        | InvocationTargetException | NoSuchMethodException |
SecurityException e) {

        System.err.println("Constructor parameter mismatched!");
    }

    /*
    * call the method and check the return type
    */

    int result1 = (int) declaredMethod.invoke(newInstance, 10, 20);
    int result2 = (int) declaredMethod.invoke(newInstance, -10, -20);
    int result3 = (int) declaredMethod.invoke(newInstance, 0, 0);

    /*
    * check the return type
    */

    if (!declaredMethod.getReturnType().equals(int.class)) {
        System.err.println("Return type not matched!");
    }

    /*
    * Test case matching
    */

    if ((result1 != 30) || (result2 != -30) || (result3 != 0)) {
        System.out.println("Test case failed");
    }

    System.out.println("Test case passed!");
}
}

```

Reflection Flow (Same Order Followed in Class)

We followed these steps one by one during the practical.

What Each Step Was Checking

Step	Validation
Load Class	Class existence
Check Class Name	Correct class name
Find Method	Method name + parameters
Create Object	Constructor
Invoke Method	Runtime execution
Check Return Type	Return type
Compare Result	Output logic

We discussed in class why this order matters, so revise it in the same sequence.

Version 1 vs Version 2

Feature	Version 1	Version 2
Class Loading	Compile Time	Runtime
Class Name Validation	No	Yes
Method Name Validation	No	Yes
Parameter Validation	No	Yes
Constructor Validation	No	Yes
Return Type Validation	No	Yes
Output Validation	Yes	Yes
Detailed Error Checking	No	Yes

Class Takeaways

- Version 1 only verifies the final result.
- Reflection verifies the program structure first, then the output.
- Every validation happens independently, making debugging easier.
- This runtime validation idea becomes useful when we start learning JUnit Testing in the next sessions.